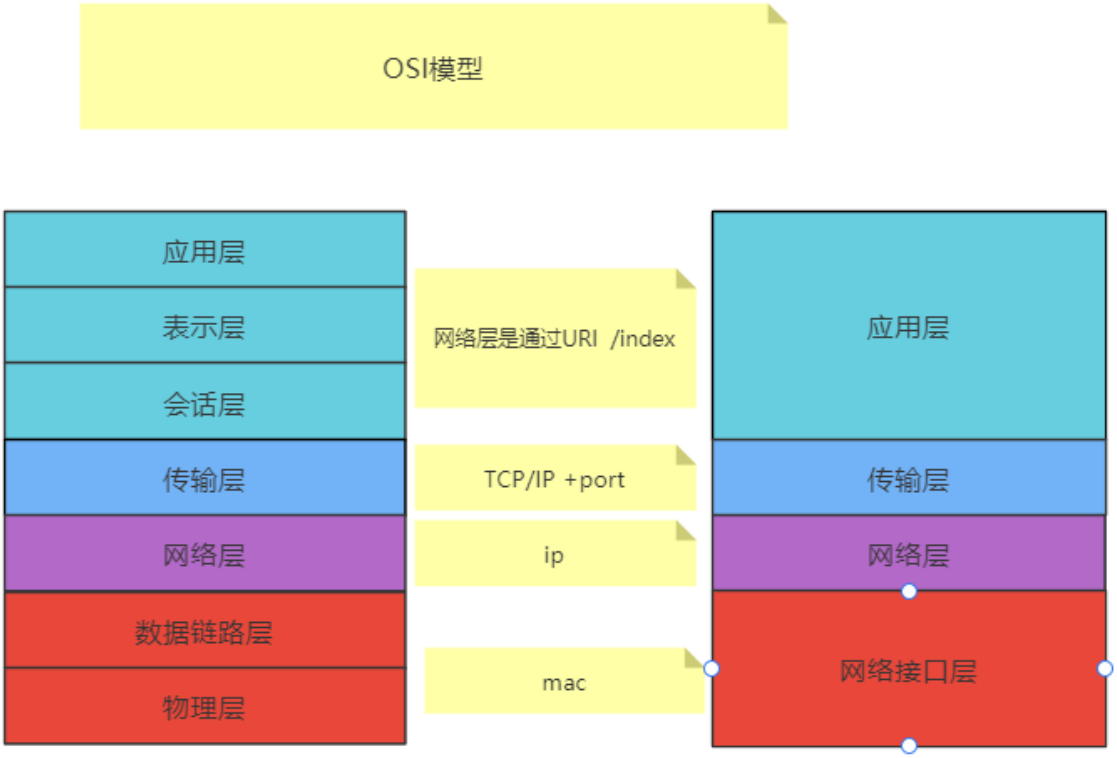
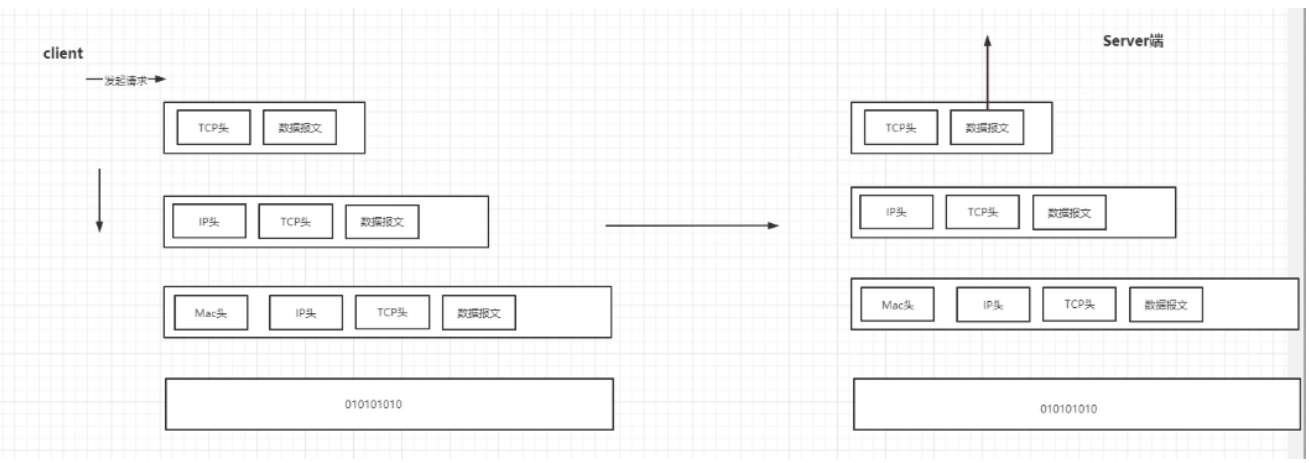
网络分层

OSI模型



可以来看下一次连接在网络中是如何传递的



1.首先根据TCP协议，把TCP头信息绑定到报文前面

2.然后由于请求中有IP,所以会绑定IP到报文前面（**ip表示我的服务器所在的位置**）

3.通过IP，发送广播，拿到对应的机器的mac地址，绑定到报文前面（**mac地址表示我的身份证明，如果只用mac地址是不知道我的服务器所在的位置的，而根据ip的前几位可以锁定目的主机的网络，然后后面的用于确定目的主机是网络中的哪一台设备**）

4.生成01010这样的二进制流传输给服务端

5.服务端在接收到二进制流后先解析

6.然后拿到mac头信息和本机的mac进行比较，看是否是同一个

7.然后在拿到ip，根据ip去比较

8.然后拿到tcp头信息，最后拿到报文信息

TCP/UDP

我们知道通信有两种协议，一种是TCP，一种是UDP，为什么我们要用TCP，很少用UDP呢，因为udp是广播性质，什么意思，就是我只管发送出去消息，至于你能不能接收我就不管了，就是这种通信是不安全的，然后TCP是通过三次握手，4次挥手进行网络通行保证的数据的安全，并不是socket只能使用TCP,他提供了两种通信的使用。

tpc的三次握手：

1.景天老师跟妹子表白（妹子可能同意，可能不同意）第一次握手

2.妹子如果同意，我跟你谈恋爱 （但是景天老师可能是渣男，可能妹子同意，他又拒绝了）第二次握手

3.如果景天老师同意了（那就正式开始谈恋爱）第三次握手

tcp的四次挥手：

1.景天老师忽然有一天跟妹子说，我们不合适，就到这里吧 第一次挥手

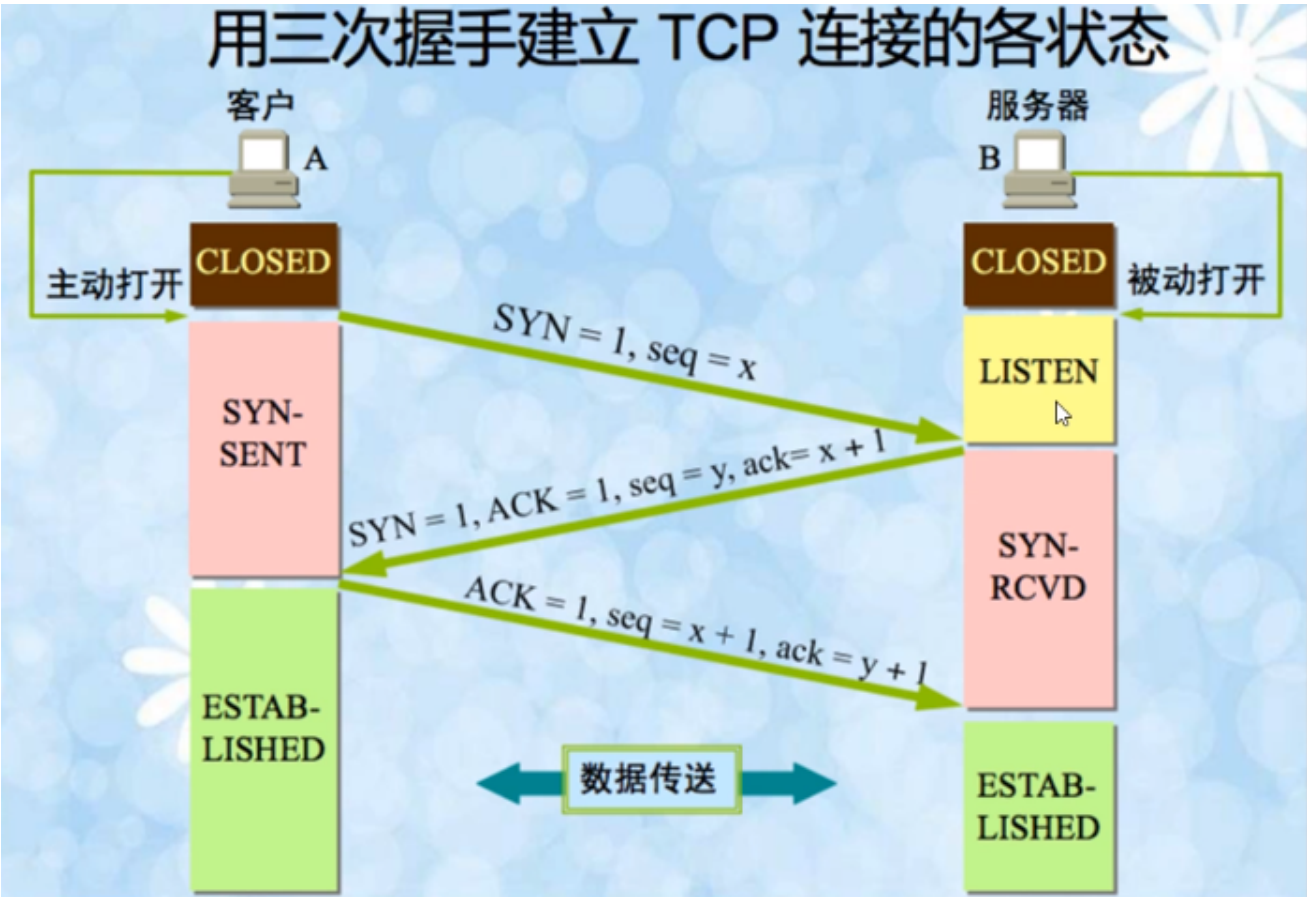
2.妹子可能还没有准备好分手，跟景天老师说等她想清楚再说 第二次挥手

3.妹子如果同意了，那么妹子就会跟景天老师说，分手吧，第三次挥手

4.景天老师得到了妹子的回复，就正式和妹子分手，签署分手协议 第四次挥手

TCP传输的可靠性

三次握手

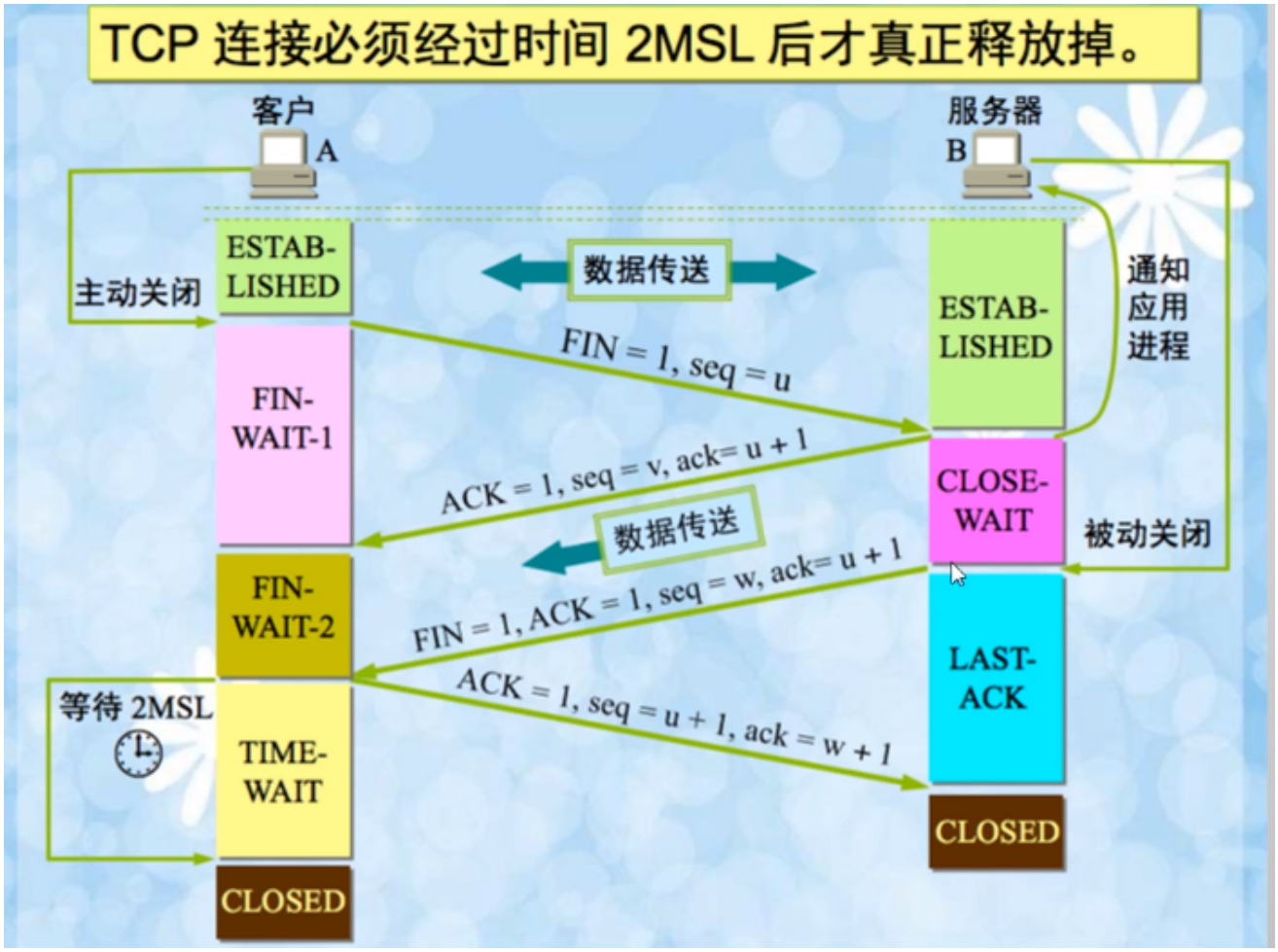


如果A服务器和B服务器建立连接

1.首先A服务器主动打开连接，把CLOSED状态改为SYN-SENT,并且向B发送数据包

2.B收到数据包后，被动的打开连接，把CLOSED状态改为LIETEN,并且返回给A一个ack的回应，告诉A我收到了，并且状态再改为SYN-RCVD

3.A收到后，把状态从SYN-SENT改为ESTAB-LISHED，会再发一个连接给B节点，因为B节点也不知道A节点现在到底有没有问题，所以再次确认下，当B收到后也把SYN-RCVD改为ESTAB-LISHED，就建立了一个双向的通道



如果客户端A和服务端B断开连接：

1.客户端A主动关闭，把状态ESTAB-LISHED改为FIN-WAIT-1,并发送消息给B，通知他被动关闭

2.如果他从返回了ack的消息，就会从ESTAB-LISHED改为CLOSE-WAIT，并且A会从FIN-WAIT-1改为FIN-WAIT-2，如果等待2MSL都没有收到消息，会改为TIME-WAIT，再直接关闭

3.如果B处理完数据后，返回一个ack的消息，B会从CLOSE-WAIT改为LAST-ACK的状态，A收到后。会改为TIME-WAIT，并且关闭

4.B收到后直接关闭

(B) --> [SYN] --> (A)

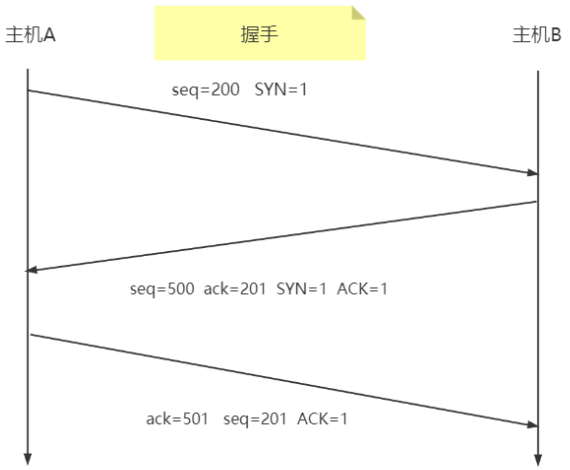
假如服务器A和客户机B通讯. 当A要和B通信时，B首先向A发一个SYN (Synchronize) 标记的包，告诉A请求建立连接

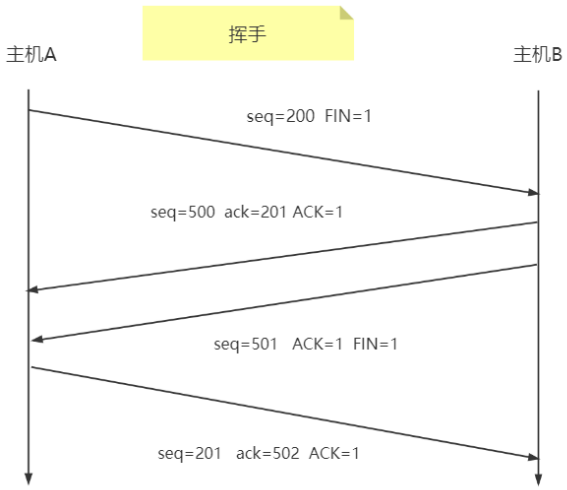
(B) <-- [SYN/ACK] <--(A)

接着，A收到后会发一个对SYN包的确认包(SYN/ACK)回去，表示对第一个SYN包的确认，并继续握手操作.

(B) --> [ACK] --> (A)

B收到SYN/ACK 包,B发一个确认包(ACK)，通知A连接已建立。至此，三次握手完成，一个TCP连接完成





MSL是什么?

MSL 是Maximum Segment Lifetime英文的缩写，中文可以译为“报文最大生存时间”，他是任何报文在网络上存在的最长时间，超过这个时间报文将被丢弃。RFC 793中规定了MSL为2分钟，实际应用中常用的是30秒，1分钟和2分钟等。

Http、Https、Tcp、Udp的区别

HTTP：本质底层还是用的TCP协议，但是HTTP属于应用层协议，属于无状态的

HTTPS:SSL+HTTP协议构建的可进行加密传输、身份认证的网络协议，比http协议安全

基于socket去实现rpc

我们都知道unix(like)世界里，一切皆文件，而文件是什么呢？文件就是一串二进制流而已，不管socket,还是FIFO、管道、终端，对我们来说，一切都是文件，一切都是流。在信息 交换的过程中，我们都是对这些流进行数据的收发操作，简称为I/O操作(input and output)，往流中读出数据，系统调用read，写入数据，系统调用write。不过话说回来了 ，计算机里有这么多的流，我怎么知道要操作哪个流呢？对，就是文件描述符，即通常所说的fd，一个fd就是一个整数，所以，对这个整数的操作，就是对这个文件（流）的操作。我们创建一个socket,通过系统调用会返回一个文件描述符，那么剩下对socket的操作就会转化为对这个描述符的操作。不能不说这又是一种分层和抽象的思想。

服务端的代码

```
public class ServerSocketDemo {

    public static void main(String[] args) {
        ServerSocket serverSocket = null;
        try {
            //localhost: 8080
            serverSocket = new ServerSocket(8080);
            Socket socket = serverSocket.accept(); //监听客户端连接(连接阻塞)
            System.out.println(socket.getPort());

        } catch (IOException e) {
            e.printStackTrace();
```

```
        } finally {
            //TODO

        }

    }
}
```

客户端的代码

```
public class SocketClientDemo {

    public static void main(String[] args) {

        try {
            Socket socket=new Socket("localhost",8080);
        } catch (IOException e) {
            e.printStackTrace();
        }

    }
}
```

启动服务端

可以看到我们启动服务端

启动客户端

可以看到服务端这边打印出端口号

服务端的阻塞

这个服务端进程并没有结束，那么他阻塞在哪里了呢？我们看下accept方法，这个就是它让连接阻塞了，打断点看下

好现在我们知道服务端和客户端已经建立通信了，那么是不是就可以互相传递数据了，那么怎么传递数据呢，我们就使用io流来进行数据的传输。

客户端和服务端如何传递数据

服务端代码：

```
 public static void main(String[] args) {
        ServerSocket serverSocket = null;
        try {
            //localhost: 8080
            serverSocket = new ServerSocket(8080);
            Socket socket = serverSocket.accept(); //监听客户端连接(连接阻塞)
            System.out.println(socket.getPort());
            // 得到客户端的输入流
```

```
            BufferedReader bufferedReader = new BufferedReader(new
InputStreamReader(socket.getInputStream()));//输入流
            String s = bufferedReader.readLine(); //被阻塞了
            String clientStr = s; //读取客户端的一行数据
            System.out.println("接收到客户端的信息: " + clientStr);
            //给客户端输出数据
            BufferedWriter bufferedWriter = new BufferedWriter(new
OutputStreamWriter(socket.getOutputStream()));
            bufferedWriter.write("我收到了信息\n");
            bufferedWriter.flush();

            bufferedReader.close();
            bufferedWriter.close();
        } catch (IOException e) {
            e.printStackTrace();
        } finally {

        }

    }
```

客户端代码:

```
 public static void main(String[] args) {

        try {
            Socket socket=new Socket("localhost",8080);
            // 给服务端发送数据
            BufferedWriter bufferedWriter=new BufferedWriter(new
OutputStreamWriter(socket.getOutputStream()));
            bufferedWriter.write("我是客户端1，发送了一个消息\n");
            bufferedWriter.flush();
            // 读取服务端的数据
            BufferedReader bufferedReader=new BufferedReader(new
InputStreamReader(socket.getInputStream()));//输入流
            String serverLine=bufferedReader.readLine(); //读取服务端返回的数据
            System.out.println("服务端返回的数据:"+serverLine);

        } catch (IOException e) {
            e.printStackTrace();
        }

    }
```

这个地方的 bufferedWriter.write("我是客户端1，发送了一个消息\n");中一定要加一个\n，因为服务端是通过readLine()读取的数据，不加这一个服务端不知道有没有读完，就卡在那里了

在BufferReader中的有一段代码,可以看到，如果没有\n结尾或者\r结尾是eol是不会为true的，后面的s永远都没有值，因为被包在了一个无限的for循环中

```
for (i = nextChar; i < nChars; i++) {
    c = cb[i];
    if ((c == '\n') || (c == '\r')) {
        eol = true;
        break charLoop;
    }
}
```

两个客户端同时访问

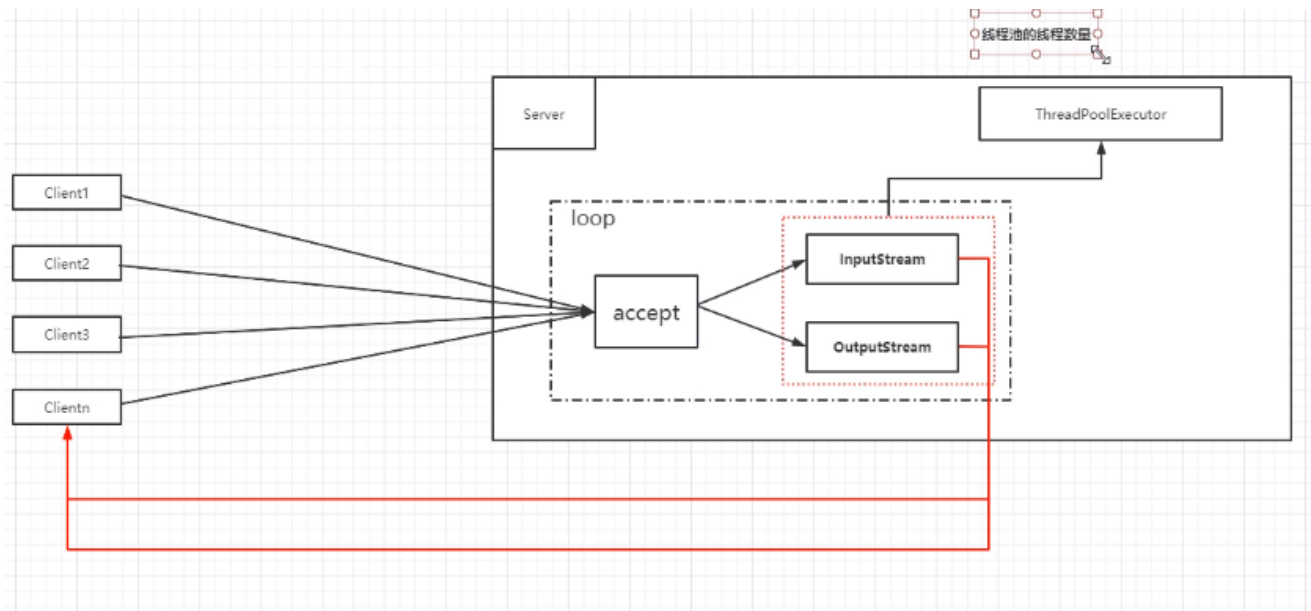
使用两个cmd的telnet 127.0.0.1 8080去访问服务端，可以看到传统io的阻塞点在哪里

- socket.accept();
- bufferedReader.readLine()

我们添加while循环

会发现好像是没有问题，可以同时多个进行访问，但是我们知道，我们的IO其实是阻塞的，一个线程去读取IO资源会阻塞在这里，如果数据量过大，就会一直阻塞在这里，效率就特别的慢，我们就想到了是不是可以用多线程，通过多线程异步的方式让IO不阻塞，为什么要用多线程，就是我们一个线程去操作IO，会阻塞，我用多个线程去操作IO，是不是就不阻塞了

所以我们就可以弄成异步是不是就不阻塞了，现在我们通过线程池的方式让他不阻塞



代码如下：新建一个SocketThread类

```
public class SocketThread implements Runnable{

    private Socket socket;

    public SocketThread(Socket socket) {
        this.socket = socket;
```

```
    }

    @Override
    public void run() {
        try {
            BufferedReader bufferedReader = new BufferedReader(new
InputStreamReader(socket.getInputStream()));//输入流
            String s = bufferedReader.readLine(); //被阻塞了
            String clientStr = s; //读取客户端的一行数据
            System.out.println("接收到客户端的信息: " + clientStr);
            //写回去
            BufferedWriter bufferedWriter = new BufferedWriter(new
OutputStreamWriter(socket.getOutputStream()));
            bufferedWriter.write("我收到了信息\n");
            bufferedWriter.flush();

            bufferedReader.close();
            bufferedWriter.close();
        }catch (Exception e){

        }
    }
}
```

服务端：添加一个线程池,并且添加一个while循环

```
 static ExecutorService executorService= Executors.newFixedThreadPool(20);

    public static void main(String[] args) {
        ServerSocket serverSocket=null;
        try {
            //localhost: 8080
            serverSocket=new ServerSocket(8080);
            while(true) {
                Socket socket = serverSocket.accept(); //监听客户端连接(连接阻塞)
                System.out.println(socket.getPort());
                executorService.execute(new SocketThread(socket)); //异步
            }
        } catch (IOException e) {
            e.printStackTrace();
        } finally {
            //TODO
            if(serverSocket!=null){
                try {
                    serverSocket.close();
                } catch (IOException e) {
                    e.printStackTrace();
                }
            }
        }
```

这种就可以同时处理多个请求了，但是我们知道线程是消耗我们系统资源的，所以线程是有限的，那么就会有问题，如果我有很多的请求怎么办？可能会有很多的请求都会被丢失

怎么去提升连接数呢?

我们刚才是因为他阻塞了，就用了线程池做异步处理，现在发现线程会消耗资源，那么现在就引出了一个非阻塞IO的概念,这个不是NIO，这个是不阻塞IO

连接阻塞->连接非阻塞

IO阻塞->IO非阻塞

那么再来看一段代码

服务端:

```
public class NoIoServer {

    public static void main(String[] args) {
        try {
            ServerSocketChannel serverSocketChannel=ServerSocketChannel.open();
             //设置为非阻塞，非阻塞IO默认是阻塞的
            serverSocketChannel.configureBlocking(false);
            serverSocketChannel.socket().bind(new InetSocketAddress(8080));
            while(true){
                // 如果不设置为非阻塞，就被卡在这个地方了
                SocketChannel socketChannel = serverSocketChannel.accept();
                if(socketChannel !=null){
                    ByteBuffer byteBuffer = ByteBuffer.allocate(1024);
                    socketChannel.read(byteBuffer);
                    System.out.println(new String(byteBuffer.array()));

                    // 写出数据，这里需要反转一下，因为我们是用的buffer缓冲区存储的，读是一个顺序，写
要反转下，写过去
                    byteBuffer.flip();
                    socketChannel.write(byteBuffer);
                }else{
                    Thread.sleep(1000);
                    System.out.println("没有建立有效的连接");
                }
            }
        } catch (IOException e) {
            e.printStackTrace();
        } catch (InterruptedException e) {
            e.printStackTrace();
        }
    }

}
```

客户端:

```
public class NoIOClient1 {
```

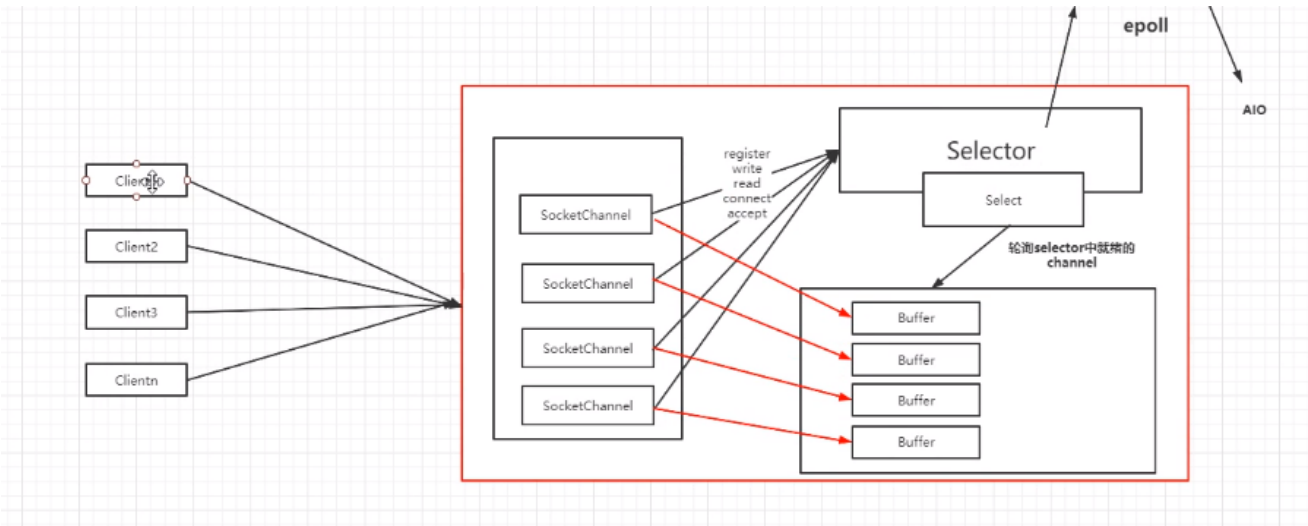
```
    public static void main(String[] args) {
        try {
            SocketChannel socketChannel=SocketChannel.open();
//            socketChannel.configureBlocking(false);
            // 如果设置为非阻塞，那么这个连接代码执行完成后，并不代表连接就已经建立好了
            socketChannel.connect(new InetSocketAddress("localhost",8080));
            if(socketChannel.isConnectionPending()){
                socketChannel.finishConnect();
            }
            ByteBuffer byteBuffer = ByteBuffer.allocate(1024);
            byteBuffer.put("保过班六期".getBytes());
            byteBuffer.flip();
            socketChannel.write(byteBuffer);
            //读取数据
            byteBuffer.clear();
            int read = socketChannel.read(byteBuffer);
            if(read>0){
                System.out.println("收到了服务端的数据"+new String(byteBuffer.array()));
            }else{
                System.out.println("没有收到数据");
            }
        } catch (IOException e) {
            e.printStackTrace();
        }
    }

}
```

多路复用机制(NIO)

如果现在有很多的用户要去买电脑，然后就去淘宝里面买电脑，淘宝里面就注册了很多的商家，会显示这个商家的状态，是否正常可以卖电脑，一当可以，那么就去卖

我们现在看下，我们其实NIO其实不是真正的异步IO



代码演示：

服务端代码：

```
public class NoIoServer {

    static Selector selector;

    public static void main(String[] args) {
        try {
            selector=Selector.open();
            //selector 必须是非阻塞
            ServerSocketChannel serverSocketChannel=ServerSocketChannel.open();
            serverSocketChannel.configureBlocking(false); //设置为非阻塞
            serverSocketChannel.socket().bind(new InetSocketAddress(8080));
            serverSocketChannel.register(selector, SelectionKey.OP_ACCEPT); //把连接事件注册
到多路复用器上
            while(true){
                selector.select(); //阻塞机制
                Set<SelectionKey> selectionKeySet=selector.selectedKeys();
                Iterator<SelectionKey> iterable=selectionKeySet.iterator();
                while(iterable.hasNext()){
                    SelectionKey key=iterable.next();
                    iterable.remove();
                    if(key.isAcceptable()){ //连接事件
                        handleAccept(key);
                    }else if(key.isReadable()){ //读的就绪事件
                        handleRead(key);
                    }
                }
            }
        } catch (IOException e) {
            e.printStackTrace();
        }
    }

    private static void handleAccept(SelectionKey selectionKey){
        //
        ServerSocketChannel serverSocketChannel=(ServerSocketChannel)
selectionKey.channel();
        try {
            // 此处因为客户端第一次连接过来，我们得到的是连接事件，建立连接后，就给客户端返回数据
            // 此处有点变化，就是我们会先给客户端响应数据，然后服务端重新轮询selector,然后得到读事件，
再读取客户端的数据
            SocketChannel socketChannel=serverSocketChannel.accept() ;//一定会有一个连接
            socketChannel.configureBlocking(false);
            socketChannel.write(ByteBuffer.wrap("Hello Client,I'm NIO
Server".getBytes()));
            // 这个地方要注册读事件，对方那边通过这个读事件去读取数据
            socketChannel.register(selector,SelectionKey.OP_READ); //注册读事件
        } catch (IOException e) {
            e.printStackTrace();
        }
    }

    private static void handleRead(SelectionKey selectionKey){
```

```
        SocketChannel socketChannel=(SocketChannel)selectionKey.channel();
        ByteBuffer byteBuffer=ByteBuffer.allocate(1024);
        try {
            socketChannel.read(byteBuffer); //这里是不是一定有值，什么情况没值呢，就是客户端只是建立连接，但是不传数据
            System.out.println("server receive msg:"+new String(byteBuffer.array()));
        } catch (IOException e) {
            e.printStackTrace();
        }

    }
}
```

客户端代码：

```
public class NewIOClient {


    static Selector selector;

    public static void main(String[] args) {
        try {
            selector=Selector.open();
            SocketChannel socketChannel=SocketChannel.open();
            socketChannel.configureBlocking(false);
            socketChannel.connect(new InetSocketAddress("localhost",8080));
            socketChannel.register(selector, SelectionKey.OP_CONNECT); //连接事件
            while(true){
                selector.select();
                Set<SelectionKey> selectionKeySet=selector.selectedKeys();
                Iterator<SelectionKey> iterator=selectionKeySet.iterator();
                while(iterator.hasNext()){
                    SelectionKey key=iterator.next();
                    iterator.remove();
                    if(key.isConnectable()){ //连接事件
                        handleConnect(key);
                    }else if(key.isReadable()){
                        handleRead(key);
                    }
                }
            }
        } catch (IOException e) {
            e.printStackTrace();
        }
    }
    private static void handleConnect(SelectionKey selectionKey) throws IOException {
        SocketChannel socketChannel=(SocketChannel)selectionKey.channel();
        if(socketChannel.isConnectionPending()){
            //
            socketChannel.finishConnect();
        }
        socketChannel.configureBlocking(false);
        socketChannel.write(ByteBuffer.wrap("Hello Server,I'm NIo Client".getBytes()));
```

```
        socketChannel.register(selector,SelectionKey.OP_READ); //
    }

    private static void handleRead(SelectionKey selectionKey) throws IOException {
        SocketChannel socketChannel=(SocketChannel)selectionKey.channel();
        //TODO
        ByteBuffer byteBuffer=ByteBuffer.allocate(1024);
        socketChannel.read(byteBuffer);
        System.out.println("client receive msg:"+new String(byteBuffer.array()));

    }

}
```

IO多路复用的底层原理

IO多路复用使用两个系统调用(select/poll/epoll和recvfrom)，blocking IO只调用了recvfrom；select/poll/epoll核心是可以同时处理多个connection，而不是更快，所以连接数不高的话，性能不一定比多线程+阻塞IO好,多路复用模型中，每一个socket，设置为non-blocking,阻塞是被select这个函数block，而不是被socket阻塞的。

1）select机制

客户端操作服务器时就会产生这三种文件描述符(简称fd)：writefds(写)、readfds(读)、和exceptfds(异常)。select会阻塞住监视3类文件描述符，等有数据、可读、可写、出异常 或超时、就会返回；返回后通过遍历fdset整个数组来找到就绪的描述符fd，然后进行对应的IO操作。

优点：　　几乎在所有的平台上支持，跨平台支持性好 缺点：　　由于是采用轮询方式全盘扫描，会随着文件描述符FD数量增多而性能下降。　　每次调用 select()，需要把 fd 集合从用户态拷贝到内核态，并进行遍历(消息传递都是从内核到用户空间)　　默认单个进程打开的FD有限制是1024个，可修改宏定义，但是效率仍然慢。

2）poll机制

基本原理与select一致，只是没有最大文件描述符限制，因为采用的是链表存储fd。

3）epoll机制

epoll之所以高性能是得益于它的三个函数　　1)epoll_create()系统启动时，在Linux内核里面申请一个B+树结构文件系统，返回epoll对象，也是一个fd　　2)epoll_ctl() 每新建一个连接，都通过该函数操作epoll对象，在这个对象里面修改添加删除对应的链接fd, 绑定一个callback函数。　　3)epoll_wait() 轮训所有的callback集合，并完成对应的IO操作

优点：　　没fd这个限制，所支持的FD上限是操作系统的最大文件句柄数，1G内存大概支持10万个句柄　　效率提高，使用回调通知而不是轮询的方式，不会随着FD数目的增加效率下降　　内核和用户空间mmap同一块内存实现

分析netty的工作流程

- 首先会创建两个线程池，一个boss线程池，一个work线程池
- boss线程池负责接收客户端的请求，work线程池接收boss给到的socketChannel真正去处理请求
- boss线程肯定是注册的accept请求，work线程肯定注册的是read请求，并且负责给客户端返回数据

1.创建两个线程池，里面分别放入boss线程和work线程，boss线程是用来接收请求，work线程是用来处理请求

- 定义boss线程的接口,将来需要拿到ServerSocketChannel，并注册OP_ACCEPT事件

```
public interface Boss {
    /**
     * 加入一个新的ServerSocket
     * @param serverChannel
     */
    public void registerAcceptChannelTask(ServerSocketChannel serverChannel);
}
```

- 定义work线程的接口，将来要去处理boss线程给到的socketChannel

```
public interface Worker {
    /**
     * 加入一个新的客户端会话
     * @param channel
     */
    public void registerNewChannelTask(SocketChannel channel);

}
```

- 由于boss和work都是线程，他们有些处理的逻辑是一样的，所以定义一个抽象类，把他们共同的逻辑写在一块，不同的逻辑就让各自的实现类去实现

```
// 因为都是boss和work线程，所以实现Runnable
public abstract class AbstractNioSelector implements Runnable {

}
```

2.每个线程都要绑定一个selector，然后要接收一个boss线程池或者work线程池，回去执行相应的任务，会添加一个阻塞的队列，来存放任务

- 在new线程的时候，就要绑定一个selector，并且执行当前的线程

```
public abstract class AbstractNioSelector implements Runnable {
    /**
     * 选择器
     */
    protected Selector selector;

    /**
     * 线程池
     */
    private final Executor executor;

    /**
```

```java
     * 任务队列
     */
    private final Queue<Runnable> taskQueue = new ConcurrentLinkedQueue<Runnable>();

    /**
     * 线程名称
     */
    private String threadName;

    AbstractNioSelector(Executor executor, String threadName) {
        this.executor = executor;
        this.threadName = threadName;
        openSelector();
    }
}

/**
* 获取selector并启动线程
*/
private void openSelector() {
    try {
        // 得到一个selector
        this.selector = Selector.open();
    } catch (IOException e) {
        throw new RuntimeException("Failed to create a selector.");
    }
    // 执行当前的线程
    executor.execute(this);
}

@Override
public void run() {
    // 设置当前的线程名称
    Thread.currentThread().setName(this.threadName);

    while (true) {
        try {
            // 执行标志位，这个东西后面回去唤醒阻塞的selector，后面再看
            wakenUp.set(false);
            // 如果是boss线程，首先会阻塞在这里，work线程的话之睡眠500ms
            select(selector);
            // 无论是boss线程还是work线程，都要去队列里面取任务，这个任务就是一个注册事件
            // 如果是boss线程，那么就是注册accept事件，如果是work线程，就是read事件
            // 但是此时是没有放任何任务的，所以相当于啥也没做
            processTaskQueue();
            // 如果是boss线程，首先从selector中取出selectedKeys，如果没有直接返回
            // 如果是work线程，首先从selector中取出selectedKeys，如果没有直接返回
            // 这里是抽象方法，等待相应的线程去实现
            process(selector);
        } catch (Exception e) {
            // ignore
        }

    }
```

```
    /**
     * 执行队列里的任务
     */
    private void processTaskQueue() {
        for (;;) {
            final Runnable task = taskQueue.poll();
            if (task == null) {
                break;
            }
            task.run();
        }
    }

    /**
     * select抽象方法
     *
     * @param selector
     * @return
     * @throws IOException
     */
    protected abstract int select(Selector selector) throws IOException;

    /**
     * selector的业务处理
     *
     * @param selector
     * @throws IOException
     */
    protected abstract void process(Selector selector) throws IOException;

}
```

4.把两个线程池分别交个一个selector管理者，并初始化boss线程和work线程

- 定义管理者的类

```
public class NioSelectorRunnablePool {

    /**
     * boss线程数组,并且定义一个原子性类，来保证并发，然后通过下标拿到相应的线程
     */
    private final AtomicInteger bossIndex = new AtomicInteger();
    private Boss[] bosses;

    /**
     * worker线程数组，并且定义一个原子性类，来保证并发，然后通过下标拿到相应的线程
     */
    private final AtomicInteger workerIndex = new AtomicInteger();
    private Worker[] workeres;

    // 通过传入的两个线程池，然后通过线程池的大小来初始化线程

    public NioSelectorRunnablePool(Executor boss, Executor worker) {
```

```java
        // 首先初始化boss线程
        initBoss(boss, 1);
        initWorker(worker, Runtime.getRuntime().availableProcessors() * 2);
    }

    /**
     * 初始化boss线程
     * @param boss
     * @param count
     */
    private void initBoss(Executor boss, int count) {
        // new一个boss线程数组
        this.bosses = new NioServerBoss[count];
        for (int i = 0; i < bosses.length; i++) {
            // boss线程池，线程名称，还有selector管理者放入到boss中,等待客户端的连接的到来
            bosses[i] = new NioServerBoss(boss, "boss thread " + (i+1), this);
        }

    }

    /**
     * 初始化worker线程
     * @param worker
     * @param count
     */
    private void initWorker(Executor worker, int count) {
        // 弄一个worker的数组
        this.workeres = new NioServerWorker[count];
        for (int i = 0; i < workeres.length; i++) {
            // 把woker线程池，当前线程名称，selector的管理者放入到worker中
            workeres[i] = new NioServerWorker(worker, "worker thread " + (i+1),
this);
        }
    }

    /**
     * 获取一个boss
     * @return
     */
    public Boss nextBoss() {
         return bosses[Math.abs(bossIndex.getAndIncrement() % bosses.length)];
    }

    /**
     * 获取一个worker
     * @return
     */
    public Worker nextWorker() {
         return workeres[Math.abs(workerIndex.getAndIncrement() % workeres.length)];

    }


}
```

- 然后定义NioServerBoss去继承抽象类AbstractNioSelector，并且实现BOSS接口，分别去实现他们的方法

```
public class NioServerBoss extends AbstractNioSelector implements Boss {
    // 把线程池传入进来，会通过它去执行当前的线程，线程的名称，还有当前的线程管理者
    public NioServerBoss(Executor executor, String threadName,
NioSelectorRunnablePool selectorRunnablePool) {
        // 直接调用父类，初始化刚才的所有的东西
        super(executor, threadName, selectorRunnablePool);
    }

    // 实现抽象类中的方法
    @Override
    protected void process(Selector selector) throws IOException {
        // 这里会拿到客户端的连接，但是此时还拿不到，boss线程会注册一个accept事件，将来有客户端请
求过来的时候，就会有key了
        Set<SelectionKey> selectedKeys = selector.selectedKeys();
        if (selectedKeys.isEmpty()) {
            return;
        }

        for (Iterator<SelectionKey> i = selectedKeys.iterator(); i.hasNext();) {
            SelectionKey key = i.next();
            i.remove();
            ServerSocketChannel server = (ServerSocketChannel) key.channel();
            // 新客户端
            SocketChannel channel = server.accept();
            // 设置为非阻塞
            channel.configureBlocking(false);
            // 通过selector管理者拿到一个worker
            Worker nextworker = getSelectorRunnablePool().nextWorker();
            // 把这个任务分给work去做
            nextworker.registerNewChannelTask(channel);

            System.out.println("新客户端链接");
        }
    }

    @Override
    protected int select(Selector selector) throws IOException {
        int select = selector.select();
        return select;
    }

    public void registerAcceptChannelTask(final ServerSocketChannel serverChannel){
         final Selector selector = this.selector;
         registerTask(new Runnable() {
            @Override
            public void run() {
                try {
                    //注册serverChannel到selector
                    serverChannel.register(selector, SelectionKey.OP_ACCEPT);
```

```
                } catch (ClosedChannelException e) {
                    e.printStackTrace();
                } catch (IOException e) {
                    e.printStackTrace();
                }
            }
        });
    }

}
```

- 去实现work的注册方法

```
public class NioServerWorker extends AbstractNioSelector implements Worker {

    // 和boss线程一样
    public NioServerWorker(Executor executor, String threadName,
NioSelectorRunnablePool selectorRunnablePool) {
        super(executor, threadName, selectorRunnablePool);
    }

    /**
     * 加入一个新的socket客户端
     */
    public void registerNewChannelTask(final SocketChannel channel){
         final Selector selector = this.selector;
         // 先弄一个注册读事件的任务放入在队列中，后面轮询的时候自己回去取
         registerTask(new Runnable() {
            @Override
            public void run() {
                try {
                    //将客户端注册到selector中
                    channel.register(selector, SelectionKey.OP_READ);
                } catch (ClosedChannelException e) {
                    e.printStackTrace();
                }
            }
        });
    }

    @Override
    protected int select(Selector selector) throws IOException {
        return selector.select(500);
    }

    @Override
    protected void process(Selector selector) throws IOException {
        Set<SelectionKey> selectedKeys = selector.selectedKeys();
        if (selectedKeys.isEmpty()) {
            return;
        }
        Iterator<SelectionKey> ite = this.selector.selectedKeys().iterator();
        while (ite.hasNext()) {
```

```
            SelectionKey key = (SelectionKey) ite.next();
            // 移除，防止重复处理
            ite.remove();

            // 得到事件发生的Socket通道
            SocketChannel channel = (SocketChannel) key.channel();

            // 数据总长度
            int ret = 0;
            boolean failure = true;
            ByteBuffer buffer = ByteBuffer.allocate(1024);
            //读取数据
            try {
                ret = channel.read(buffer);
                failure = false;
            } catch (Exception e) {
                // ignore
            }
            //判断是否连接已断开
            if (ret <= 0 || failure) {
                key.cancel();
                System.out.println("客户端断开连接");
            }else{
                 System.out.println("收到数据:" + new String(buffer.array()));

                //回写数据
                ByteBuffer outBuffer = ByteBuffer.wrap("收到\n".getBytes());
                channel.write(outBuffer);// 将消息回送给客户端
            }
        }
    }

}
```

- 在抽象类中实现registerTask方法，boss和work都是一样，把这个放入到队列里面

```
/**
     * 注册一个任务并激活selector
     *
     * @param task
     */
protected final void registerTask(Runnable task) {
    // 把任务放到阻塞队列里面
    taskQueue.add(task);

    Selector selector = this.selector;

    if (selector != null) {
        // 如果当前wakenUp值等于false，则把wakenUp改为true，首先就是设置为fasle，后面会通过这个
然后调用selector.wakeup()，去激活当时阻塞的select()
        if (wakenUp.compareAndSet(false, true)) {
            // 激活阻塞的selector
            selector.wakeup();
```

```
        }
    } else {
        // 如果没有selector，则移除这个任务，因为没有selector，那么没有什么谁去服务这个任务
        taskQueue.remove(task);
    }
}
```

3.然后定义一个服务类，用来绑定端口

- 里面会去拿到boss线程去调度

```
public class ServerBootstrap {

    private NioSelectorRunnablePool selectorRunnablePool;

    public ServerBootstrap(NioSelectorRunnablePool selectorRunnablePool) {
        this.selectorRunnablePool = selectorRunnablePool;
    }

    /**
     * 绑定端口，这个就是NIO的api了
     * @param localAddress
     */
    public void bind(final SocketAddress localAddress){
        try {
            // 获得一个ServerSocket通道
            ServerSocketChannel serverChannel = ServerSocketChannel.open();
            // 设置通道为非阻塞
            serverChannel.configureBlocking(false);
            // 将该通道对应的ServerSocket绑定到port端口
            serverChannel.socket().bind(localAddress);

            //获取一个boss线程
            Boss nextBoss = selectorRunnablePool.nextBoss();
            //向boss注册一个ServerSocket通道
            nextBoss.registerAcceptChannelTask(serverChannel);
        } catch (Exception e) {
            e.printStackTrace();
        }
    }
}
```

- 当时BOSS接口里面的注册任务还没有实现，在NioServerBoss中去实现

```
public void registerAcceptChannelTask(final ServerSocketChannel serverChannel){
        // 这个地方就是在里面注册OP_ACCEPT事件，后面会去激活selector,然后队列里面有任务了，就可以去执行了
         final Selector selector = this.selector;
         registerTask(new Runnable() {
            @Override
            public void run() {
                try {
                    //注册serverChannel到selector
```

```
                    serverChannel.register(selector, SelectionKey.OP_ACCEPT);
                } catch (ClosedChannelException e) {
                    e.printStackTrace();
                } catch (IOException e) {
                    e.printStackTrace();
                }
            }
        });
    }
```

4.然后调用的时候

```
public class Start {

    public static void main(String[] args) {


        //初始化线程，里面管理者boss线程池和worker线程池，这个地方初始化的线程都一直处于空循环，啥都没有
        NioSelectorRunnablePool nioSelectorRunnablePool = new NioSelectorRunnablePool(Executors.newCachedThreadPool(), Executors.newCachedThreadPool());

        //获取服务类
        ServerBootstrap bootstrap = new ServerBootstrap(nioSelectorRunnablePool);

        //绑定端口
        bootstrap.bind(new InetSocketAddress(10101));

        System.out.println("start");
    }

}
```